

Modules and packages

Every file is a module. `use` brings another one into view.

```
src/  
  main.spf  
  shapes.spf
```

`shapes.spf` :

```
module shapes  
  
pub fn area_of_square(side: Num) -> Num => side * side  
  
fn secret_helper() -> Int => 42
```

`main.spf` :

```
use std.io  
use shapes  
  
fn main() {  
  io.say("{shapes.area_of_square(3.0)}")  
}
```

`pub` marks what the outside world may use. Everything else stays private to the file, so you can rearrange the inside freely.

The standard library

Modules under `std` come with the language:

`io` `math` `fs` `path` `time` `rand` `sys` `json` `csv` `hash` `net` `http` `thread` `task` `db` `tensor` `ui` `draw`, plus `core`, `collections`, `testing` and `ai` which are written in SPRFST itself.

```
use std.math  
use std.io  
  
fn main() {  
  io.say("{math.sqrt(144.0)}")  
  io.say("{math.clamp(42.0, 0.0, 10.0)}")  
}
```

Forge, the package manager

Forge has no central registry. A package is a folder with a `project.sprfst` and some source. You add one by path or by git URL:

```
sprfst add maths --path=../maths
sprfst add colour --git=https://example.com/colour.git
sprfst install
sprfst remove colour
```

`add` copies the package into `packages/`, works out a digest of its contents and writes it to `forge.lock`. `install` re-fetches everything in the lockfile and refuses to continue if a digest has changed, so a package cannot be swapped underneath you.

To share your own work:

```
sprfst package
```

That writes `name-version.forge`, a single file with every source and a digest of the whole thing.

Exercise

Split a program you have already written into two files, with the calculations in their own module and only the parts you need marked `pub`.